ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА МОСКВЫ
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ГОРОДА МОСКВЫ
«Колледж связи № 54» имени П.М. Вострухина

ОТЧЕТ О ВЫПОЛНЕНИИ

по дисциплине

“Внедрение и поддержка компьютерных систем”

Практической работы №5

«Интеграция Flask и SQLite (добавление данных)»

Выполнил: ст.гр. 3-ИСП11-43

ФИО: Юхимчук Евгений Викторович

Специальность: 09.02.07

Информационные системы и программирование

Проверил: преподаватель Кандыбаева

Москва

2026

Практическая работа №5. Интеграция Flask и SQLite (добавление данных)

Ссылка на репозиторий: https://github.com/gangdexyuhi/client-server-practice
Папка работы: practice_05_client_server/

Цель работы

Научиться принимать данные из HTML-формы методом POST, сохранять их в базу SQLite, проверять ввод на стороне сервера и применять паттерн Post/Redirect/Get для защиты от повторной отправки формы.

Используемые технологии

- Python 3.13, Flask 3.1: request.method, request.form, redirect, url_for
- HTML-формы (method="POST", атрибут required)
- SQLite: параметризованный INSERT
- Паттерн Post/Redirect/Get (PRG), HTTP-коды 200 и 302

Структура проекта

```
src/
├── init_db.py, check_db.py, crud_console.py
├── app.py                # GET - список, POST - добавление задачи
├── app.db
├── requirements.txt
└── templates/
    └── index.html        # форма + список + блок ошибки
```

Описание реализации

Маршрут / объявлен с methods=['GET', 'POST']. При GET выводится страница со списком, при POST функция читает поле title через request.form.get('title', '').strip(). Если значение пустое, возвращается та же страница с сообщением об ошибке — это серверная проверка, которая работает даже при обходе атрибута required. Иначе выполняется INSERT с плейсхолдером ?, commit и redirect(url_for('index')).

```
@app.route('/', methods=['GET', 'POST'])
def index():
    # ---------- POST: пришла форма ----------
    if request.method == 'POST':
        title = request.form.get('title', '').strip()

        # проверка на сервере: required в HTML можно обойти
        if not title:
            return render_template(
                'index.html',
                items=fetch_all_tasks(),
                error='Название задачи не может быть пустым.'
            )

        conn = get_connection()
        cursor = conn.cursor()
        cursor.execute("INSERT INTO tasks (title) VALUES (?)", (title,))
        conn.commit()
        conn.close()

        # Post/Redirect/Get: после добавления перенаправляем на GET,
        # чтобы F5 не отправлял форму повторно
        return redirect(url_for('index'))

    # ---------- GET: показываем страницу ----------
    return render_template('index.html', items=fetch_all_tasks())
```

Функция fetch_all_tasks() вынесена отдельно, потому что список задач нужен и при обычном GET, и при выводе ошибки после POST. Форма в шаблоне отправляет данные на {{ url_for('index') }}, имя поля title совпадает с ключом в request.form.

Ход работы и результаты запуска

1. Открыл главную страницу — над пустым списком появилась форма добавления задачи.

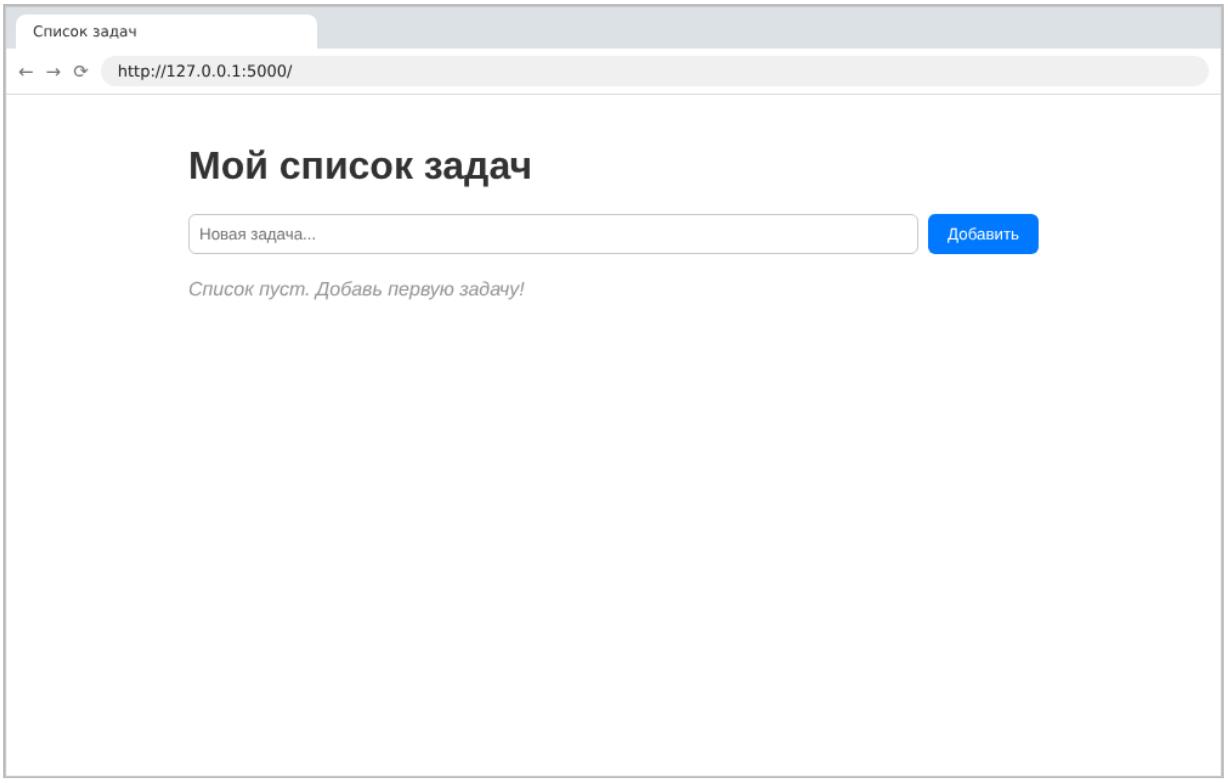


Рисунок 1 — Форма добавления задачи

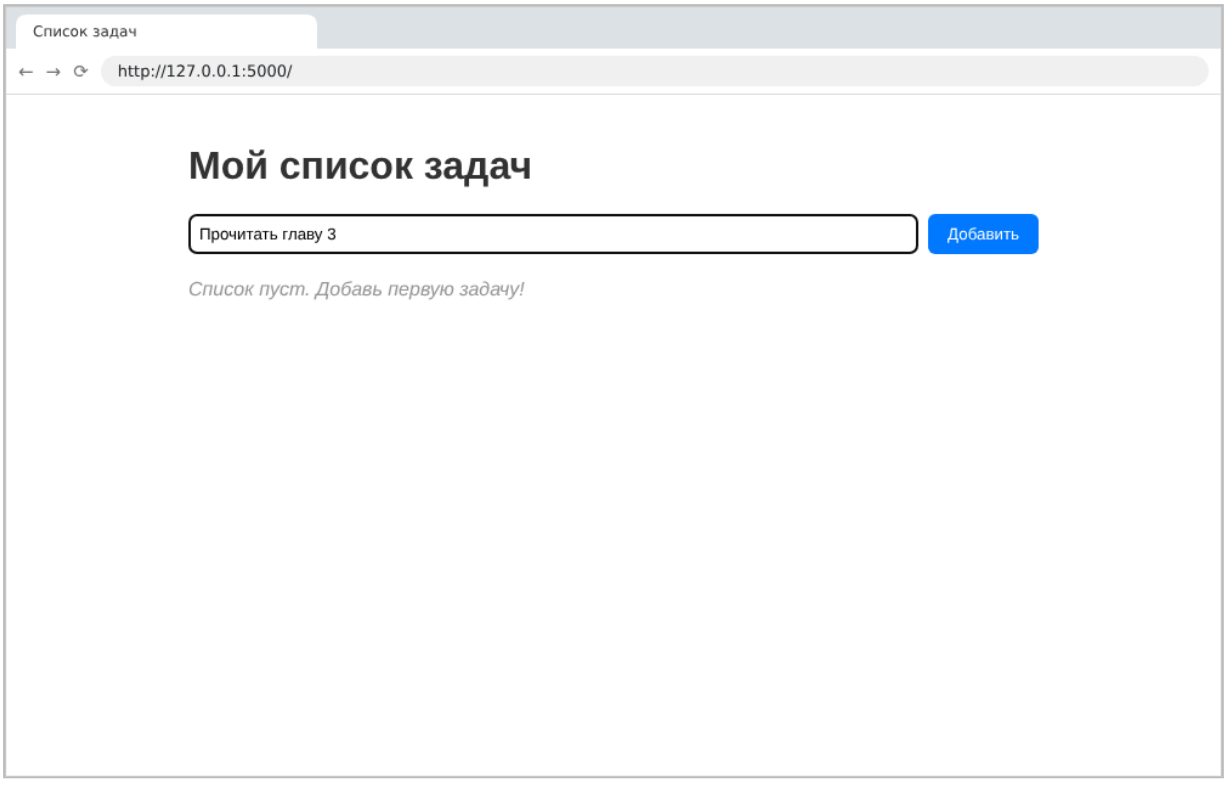


Рисунок 2 — Ввод названия задачи

2. Добавил через форму три задачи. После каждой отправки происходит POST → INSERT → редирект → GET, задача появляется в списке.

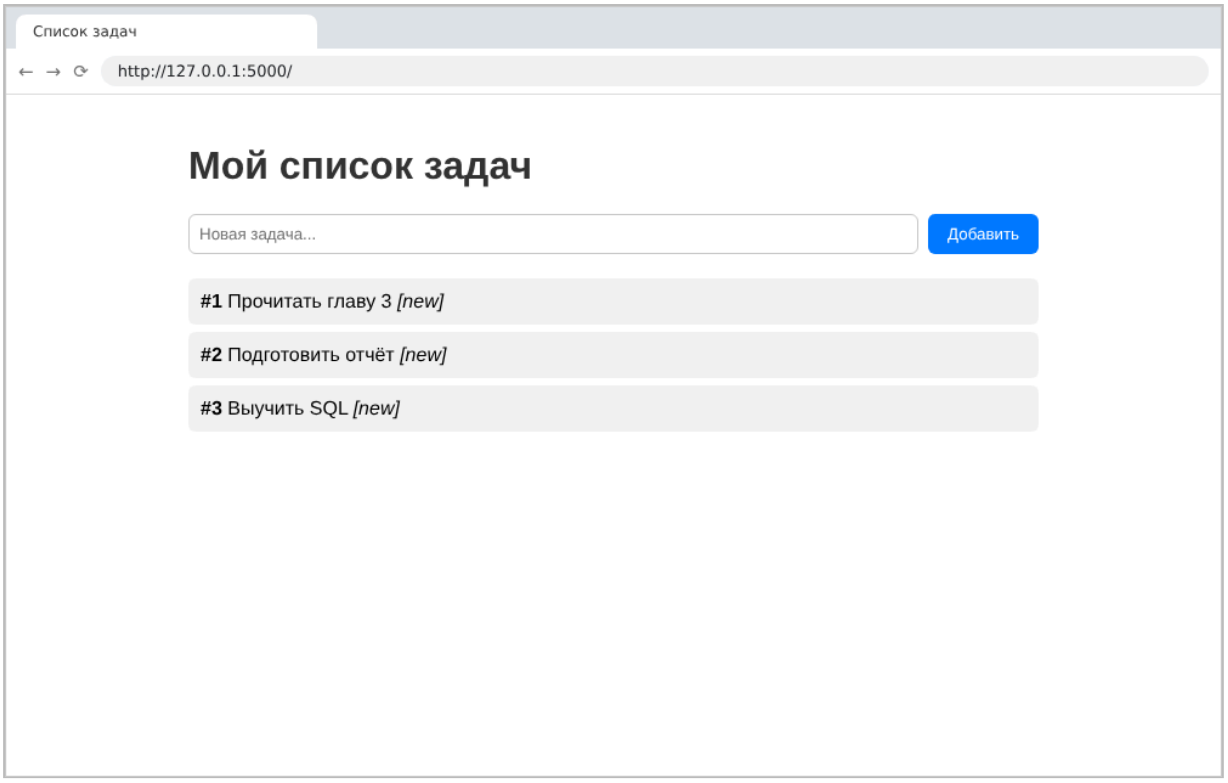


Рисунок 3 — Задачи, добавленные через форму

3. Несколько раз обновил страницу (F5) — задачи не задублировались, потому что после редиректа браузер повторяет GET, а не POST.

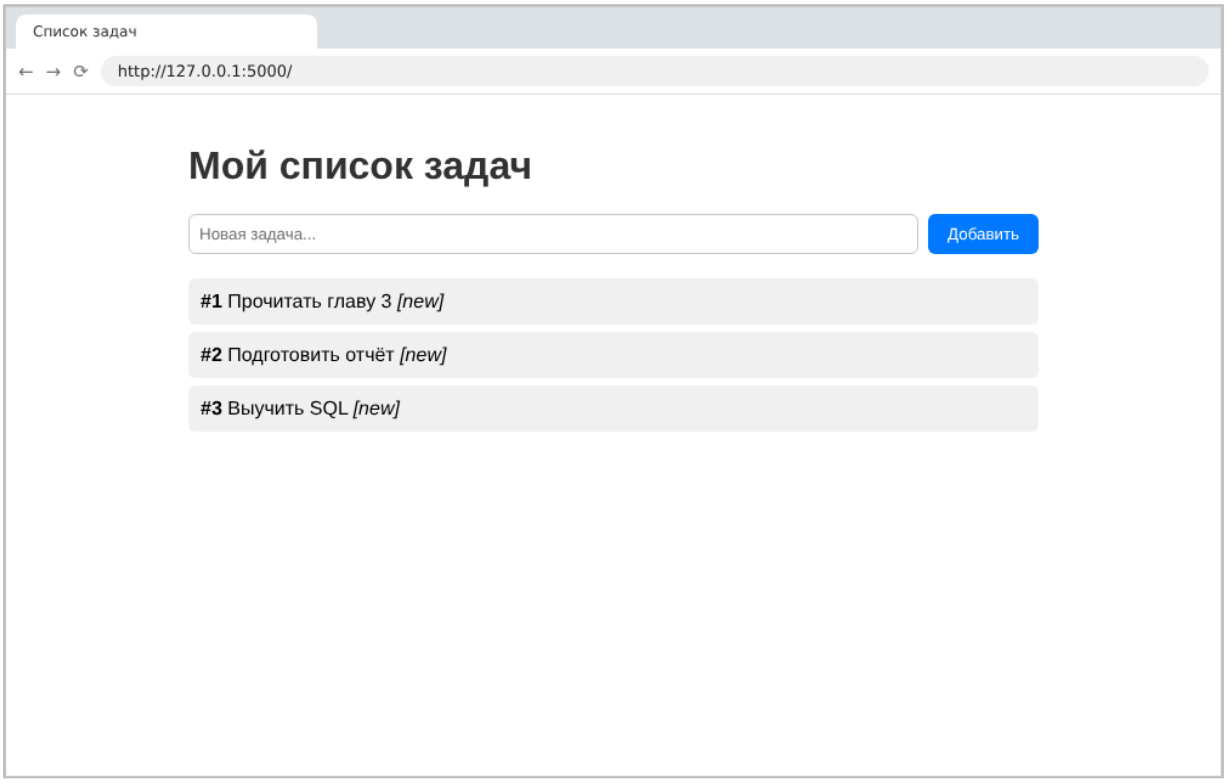


Рисунок 4 — Список после обновления страницы — дубликатов нет

4. Проверил серверную валидацию: через инструменты разработчика удалил атрибут required у поля и отправил строку из пробелов. Сервер не добавил задачу и вернул сообщение «Название задачи не может быть пустым».

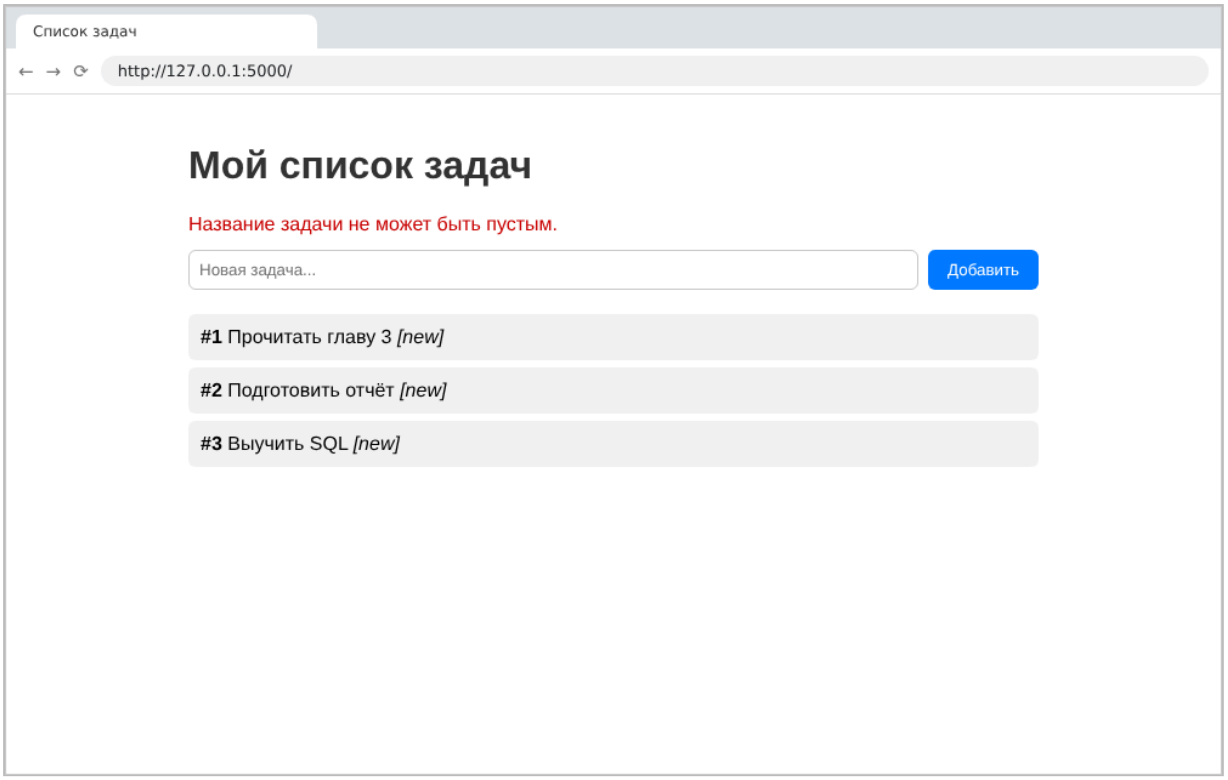


Рисунок 5 — Ошибка серверной валидации

5. Добавил задачу с названием '); DROP TABLE tasks; -- . Она сохранилась как обычный текст, таблица цела.

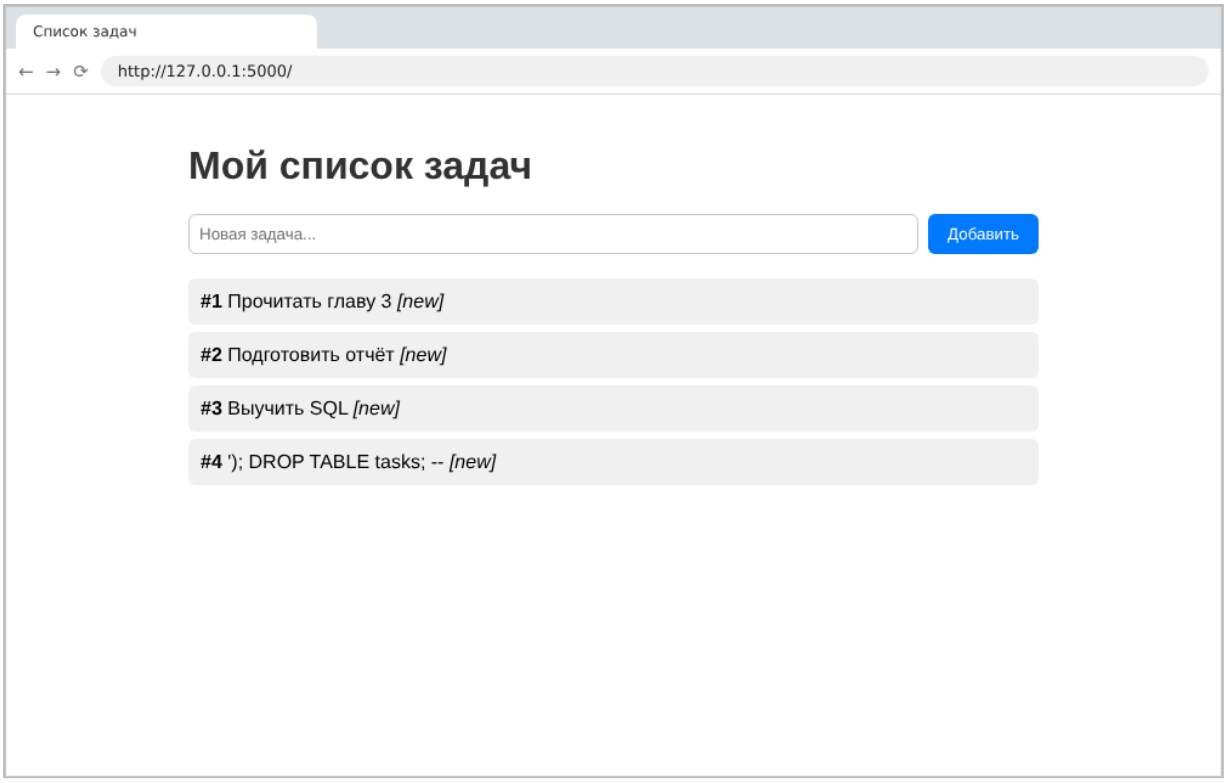


Рисунок 6 — Попытка SQL-инъекции сохранена как текст

6. Остановил сервер и запустил снова — все задачи на месте, значит commit записал их в файл app.db. В журнале сервера видна пара запросов POST / 302 и GET / 200 — это и есть PRG.

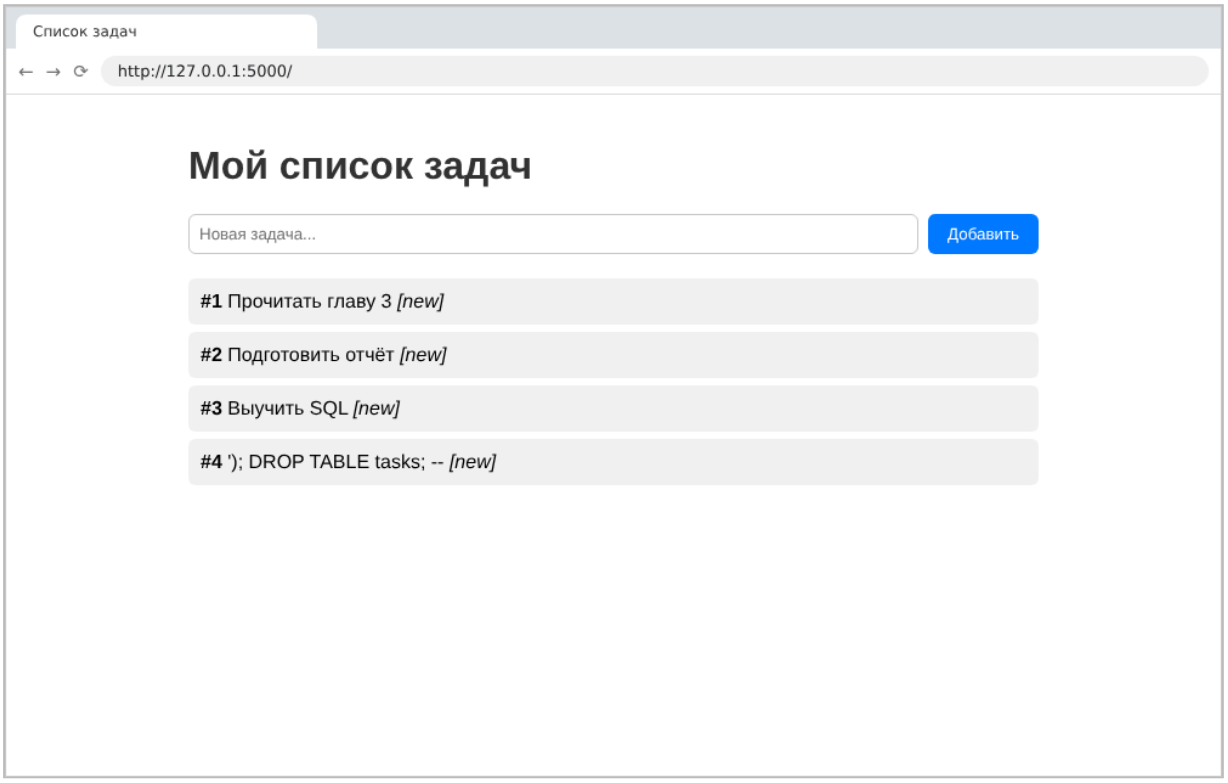


Рисунок 7 — Список задач после перезапуска сервера

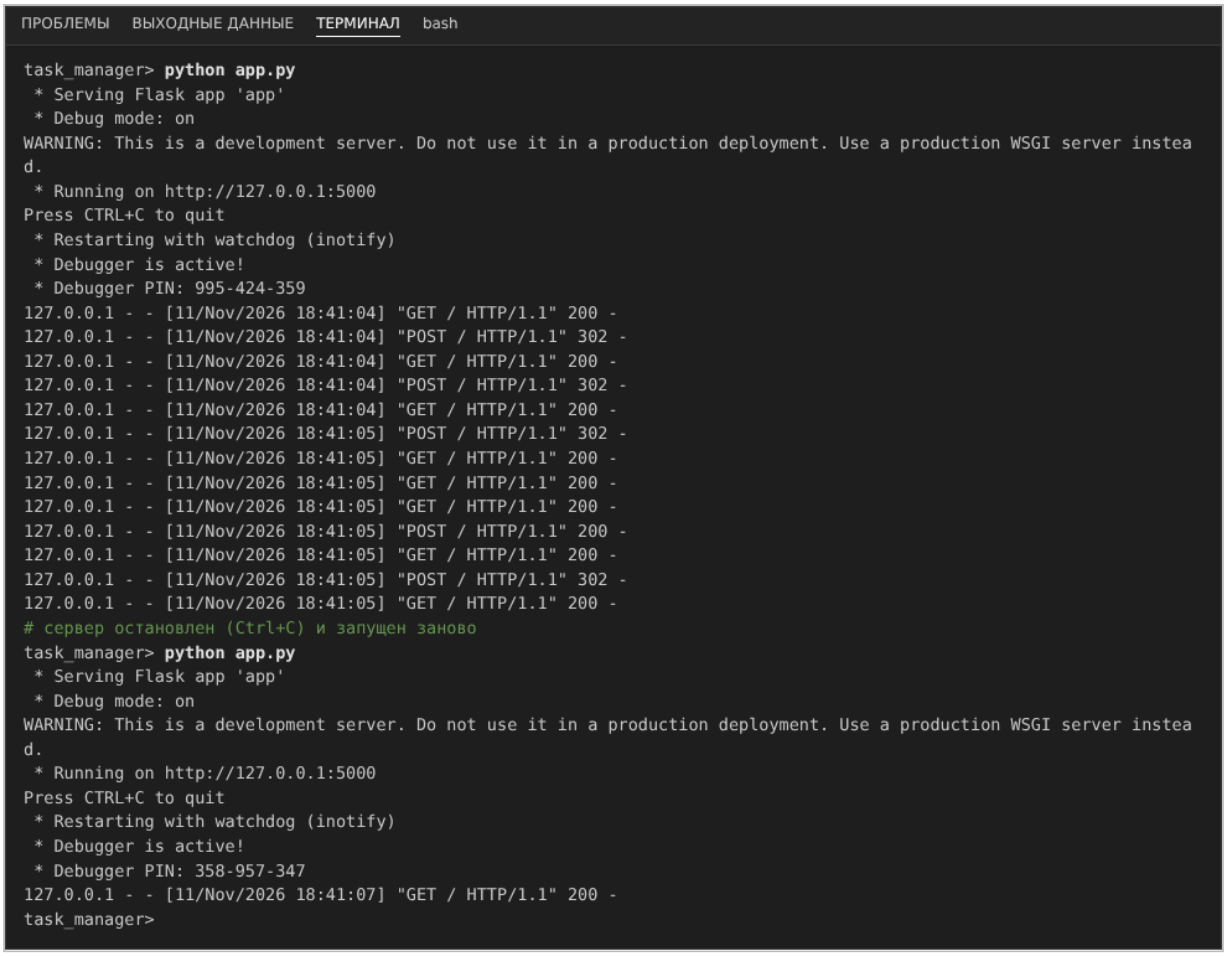


Рисунок 8 — Журнал сервера: POST с ответом 302 и последующий GET

Примеры запросов и ответов

Таблица 1 — Примеры HTTP-запросов и ответов сервера

Запрос	**Код ответа**	**Заголовок Location**	**Результат**
GET /	200 OK	—	Страница с формой и списком
POST / form: title='Новая задача'	302 FOUND	/	Задача добавлена, браузер перенаправлен на GET /
POST / form: title=' '	200 OK	—	Задача не добавлена, страница с сообщением об ошибке

Ответы на контрольные вопросы

1. Чем POST принципиально отличается от GET и почему формы отправляют именно POST?

GET предназначен для получения данных и не должен ничего менять на сервере, его параметры видны в URL, а запрос можно безопасно повторять. POST передаёт данные в теле запроса и используется для

операций, изменяющих данные. Форма добавления создаёт новую запись, поэтому она отправляется методом POST.

2. Что произойдёт, если убрать methods=['GET', 'POST'] из маршрута?

Маршрут будет принимать только GET, и при отправке формы сервер вернёт ошибку 405 Method Not Allowed (это видно в работе №4, где POST / давал 405).

3. Зачем нужен паттерн Post/Redirect/Get? Что сломается без него?

После успешного POST сервер отвечает редиректом (302) на GET-страницу. Без него браузер запоминает последний запрос как POST, и при обновлении страницы (F5) форма отправляется повторно — задача дублируется.

4. Почему url_for('index') лучше, чем жёсткая строка '/'?

url_for строит адрес по имени функции-обработчика. Если URL маршрута изменится, все ссылки и редиректы обновятся автоматически, а жёстко прописанная строка останется старой и сломается.

5. Почему request.form.get('title', '') безопаснее, чем request.form['title']?

Если поля title в запросе нет, request.form['title'] выбросит ошибку BadRequestKeyError (ответ 400), а get вернёт значение по умолчанию — пустую строку, которую затем отсеет проверка.

6. Зачем нужна серверная валидация, если в HTML уже есть required?

Атрибут required проверяет поле только в браузере, и его легко обойти: удалить атрибут через инструменты разработчика или отправить запрос напрямую. Сервер должен сам проверять данные, что и показал эксперимент со строкой из пробелов.

7. В каком порядке идут транзакции при добавлении задачи через форму?

Браузер отправляет POST → Flask читает и проверяет title → открывает соединение → выполняет INSERT с параметром → commit → закрывает соединение → отвечает редиректом 302 → браузер выполняет GET / → сервер читает задачи и возвращает страницу.

Вывод

В ходе работы я добавил в приложение форму создания задачи и обработку POST-запросов. Я реализовал серверную валидацию, применил паттерн Post/Redirect/Get и убедился, что обновление страницы не дублирует данные, задачи сохраняются после перезапуска сервера, а SQL-инъекция не срабатывает благодаря параметризованному запросу.